

Data Visualization Lab Manual (Python & R)

1. Scenario Title: Smart City Air Quality Monitoring

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import TwoSlopeNorm, ListedColormap

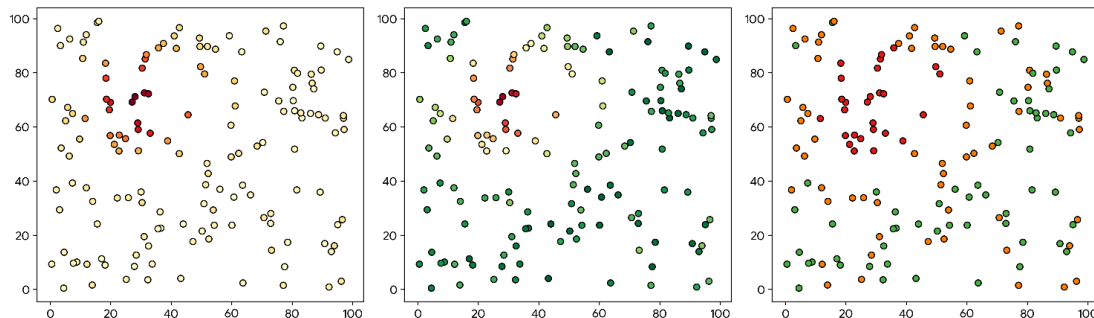
x, y = np.random.uniform(0, 100, 150), np.random.uniform(0, 100, 150)
pm25 = np.clip(10 + 80 * np.exp(-((x-30)**2 + (y-70)**2)/400) + np.random.normal(0, 5, 150), 5, 120)
categories = np.where(pm25 <= 12, 0, np.where(pm25 <= 35.4, 1, 2))

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
axes[0].scatter(x, y, c=pm25, cmap='YlOrRd', edgecolors='k')
norm = TwoSlopeNorm(vcenter=35.4, vmin=pm25.min(), vmax=pm25.max())
axes[1].scatter(x, y, c=pm25, cmap='RdYlGn_r', norm=norm, edgecolors='k')
cmap_q = ListedColormap(['#4daf4a', '#ff7f00', '#e41a1c'])
axes[2].scatter(x, y, c=categories, cmap=cmap_q, edgecolors='k')
plt.tight_layout()
plt.savefig('p1.png', dpi=200)
plt.close()
```

R Code:

```
library(ggplot2); library(patchwork)
df <- data.frame(x=runif(150,0,100), y=runif(150,0,100))
df$pm25 <- pmax(10 + 80*exp(-((df$x-30)^2+(df$y-70)^2)/400) + rnorm(150,0,5), 5)
df$cat <- cut(df$pm25, breaks=c(-Inf, 12, 35.4, Inf), labels=c("G","M","U"))
p1 <- ggplot(df, aes(x,y,fill=pm25)) + geom_point(shape=21) + scale_fill_distiller(palette="YlOrRd",
direction=1)
p2 <- ggplot(df, aes(x,y,fill=pm25)) + geom_point(shape=21) + scale_fill_gradient2(low="green",
mid="yellow", high="red", midpoint=35.4)
p3 <- ggplot(df, aes(x,y,fill=cat)) + geom_point(shape=21) +
scale_fill_manual(values=c("green","orange","red"))
p1 + p2 + p3
```

Python Output Graph for the Scenario:



2. Scenario Title: Earthquake Risk Analysis

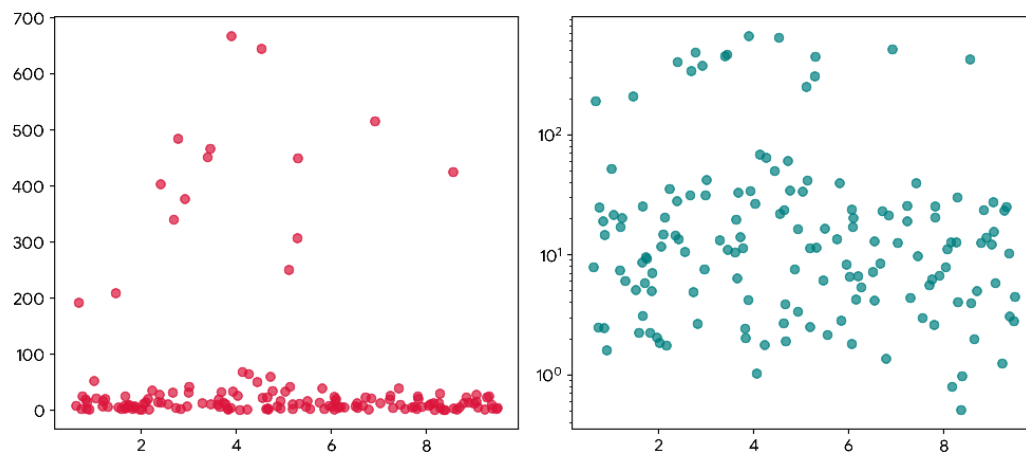
Python Code:

```
magnitudes = np.random.uniform(0.5, 9.5, 150)
depths = np.concatenate([np.random.exponential(15, 135), np.random.uniform(100, 700, 15)])
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
axes[0].scatter(magnitudes, depths, alpha=0.7, color='crimson')
axes[1].scatter(magnitudes, depths, alpha=0.7, color='teal')
axes[1].set_yscale('log')
plt.tight_layout()
plt.savefig('p2.png', dpi=200)
plt.close()
```

R Code:

```
df_eq <- data.frame(mag = runif(150, 0.5, 9.5), depth = rexp(150, 0.02))
p1 <- ggplot(df_eq, aes(mag, depth)) + geom_point(color="darkred")
p2 <- ggplot(df_eq, aes(mag, depth)) + geom_point(color="teal") + scale_y_log10()
p1 + p2
```

Python Output Graph for the Scenario:



3. Scenario Title: Wind Direction Analysis for Airport Operations

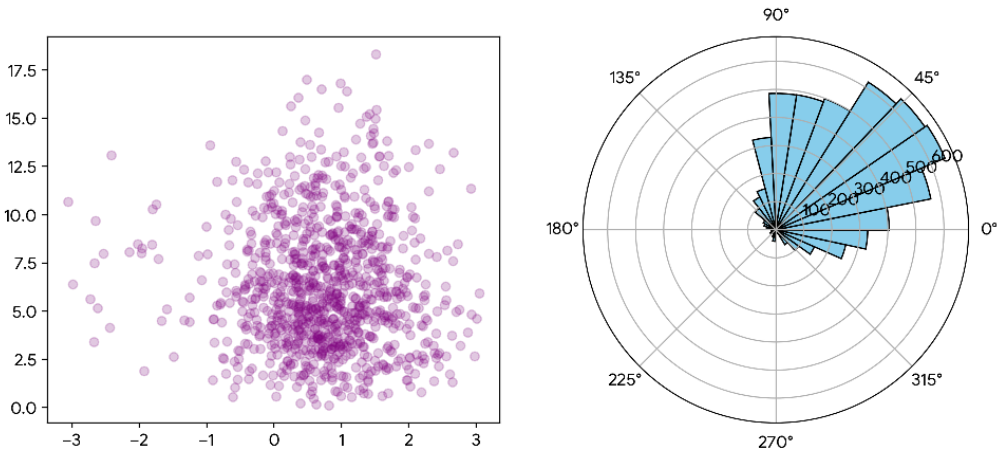
Python Code:

```
angles = np.random.vonmises(np.pi/4, 2, 1000)
speeds = np.random.rayleigh(5, 1000)
fig = plt.figure(figsize=(10, 4.5))
ax1 = fig.add_subplot(121)
ax1.scatter(angles, speeds, alpha=0.2, color='purple')
ax2 = fig.add_subplot(122, polar=True)
ax2.hist(angles, bins=30, weights=speeds, color='skyblue', edgecolor='k')
plt.savefig('p3.png', dpi=200)
plt.close()
```

R Code:

```
df_wind <- data.frame(angle = rnorm(500, 45, 20), speed = rgamma(500, 2, 0.2))
p1 <- ggplot(df_wind, aes(angle, speed)) + geom_point(color="purple", alpha=0.3)
p2 <- ggplot(df_wind, aes(x=angle)) + geom_histogram(binwidth=15, fill="skyblue") + coord_polar()
p1 + p2
```

Python Output Graph for the Scenario:



4. Scenario Title: Hospital Patient Monitoring Dashboard

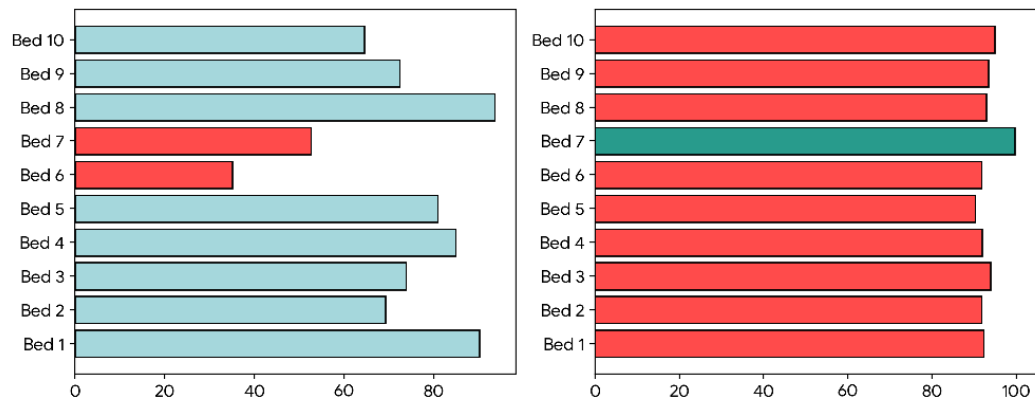
Python Code:

```
patients = [f"Bed {i+1}" for i in range(10)]
hr = np.random.normal(75, 20, 10)
spo2 = np.random.uniform(88, 100, 10)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
colors_hr = ['#ff4d4d' if (x < 60 or x > 100) else '#a8dadc' for x in hr]
axes[0].barh(patients, hr, color=colors_hr, edgecolor='black')
colors_spo2 = ['#ff4d4d' if x < 95 else '#2a9d8f' for x in spo2]
axes[1].barh(patients, spo2, color=colors_spo2, edgecolor='black')
plt.savefig('p4.png', dpi=200)
plt.close()
```

R Code:

```
df_hosp <- data.frame(bed=factor(1:10), hr=rnorm(10, 75, 20), spo2=runif(10, 88, 100))
p1 <- ggplot(df_hosp, aes(hr, bed, fill=hr>100 | hr<60)) + geom_col() +
  scale_fill_manual(values=c("#a8dadc", "#ff4d4d"))
p2 <- ggplot(df_hosp, aes(spo2, bed, fill=spo2<95)) + geom_col() +
  scale_fill_manual(values=c("#2a9d8f", "#ff4d4d"))
p1 + p2
```

Python Output Graph for the Scenario:



5. Scenario Title: Global Climate Change Study

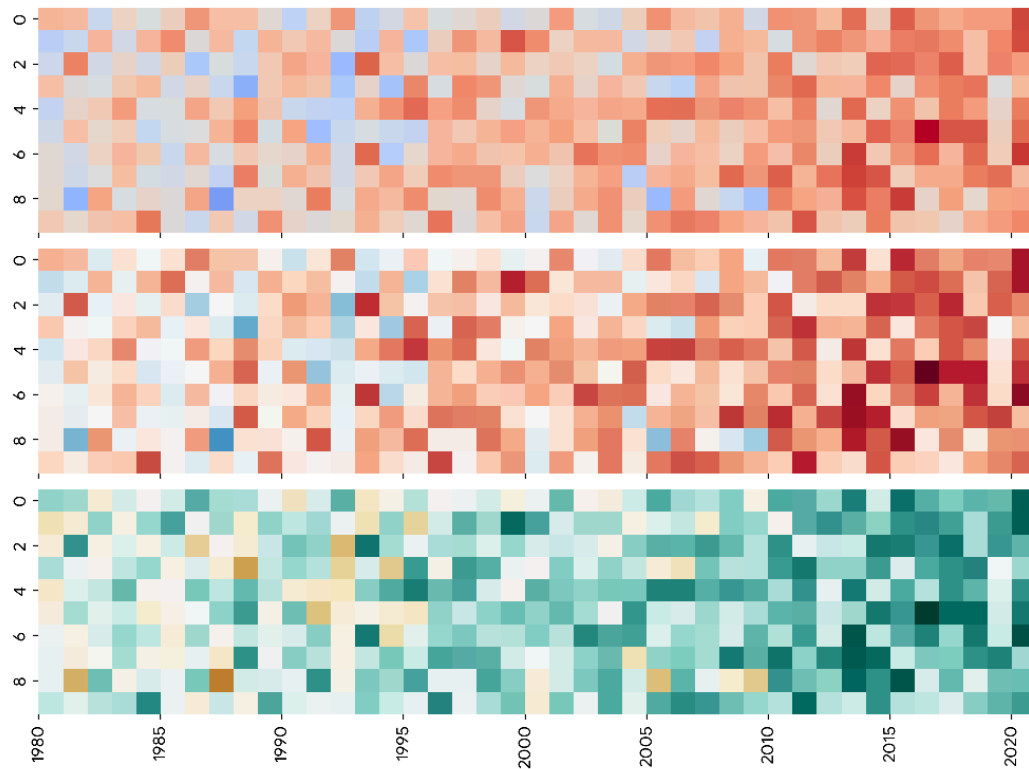
Python Code:

```
years = np.arange(1980, 2021)
anomalies = np.random.normal(0.3, 0.5, size=(10, len(years))) + np.linspace(-0.2, 0.7, len(years))
fig, axes = plt.subplots(3, 1, figsize=(10, 7.5))
sns.heatmap(anomalies, cmap='coolwarm', center=0, ax=axes[0], cbar=False)
sns.heatmap(anomalies, cmap='RdBu_r', center=0, ax=axes[1], cbar=False)
sns.heatmap(anomalies, cmap='BrBG', center=0, ax=axes[2], cbar=False)
plt.savefig('p5.png', dpi=200)
plt.close()
```

R Code:

```
matrix_data <- expand.grid(Year=1980:2020, Country=factor(1:10))
matrix_data$anomaly <- rnorm(nrow(matrix_data), 0.4, 0.5)
p1 <- ggplot(matrix_data, aes(Year, Country, fill=anomaly)) + geom_tile() + scale_fill_distiller(palette="RdBu")
p2 <- ggplot(matrix_data, aes(Year, Country, fill=anomaly)) + geom_tile() +
  scale_fill_distiller(palette="PRGn")
p1 + p2
```

Python Output Graph for the Scenario:



6. Scenario Title: E-Commerce Sales Analytics

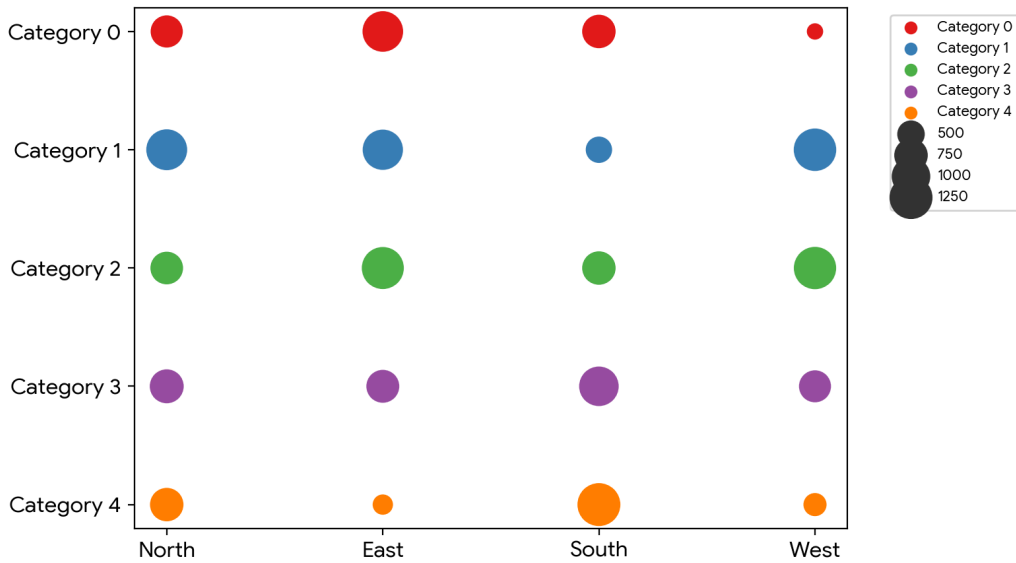
Python Code:

```
regions = ['North', 'East', 'South', 'West'] * 5
categories = np.repeat([f'Category {i}' for i in range(5)], 4)
sales_volume = np.random.randint(100, 1500, 20)
fig, ax = plt.subplots(figsize=(8, 4.5))
sns.scatterplot(x=regions, y=categories, size=sales_volume, hue=categories, palette='Set1', sizes=(100, 600), ax=ax)
plt.savefig('p6.png', dpi=200)
plt.close()
```

R Code:

```
df_sales <- expand.grid(Region=c("N","E","S","W"), Category=paste("Cat", 1:5))
df_sales$Volume <- runif(nrow(df_sales), 100, 1000)
ggplot(df_sales, aes(Region, Category, color=Category, size=Volume)) + geom_point() +
  scale_color_brewer(palette="Set1")
```

Python Output Graph for the Scenario:



7. Scenario Title: Satellite Image Vegetation Analysis

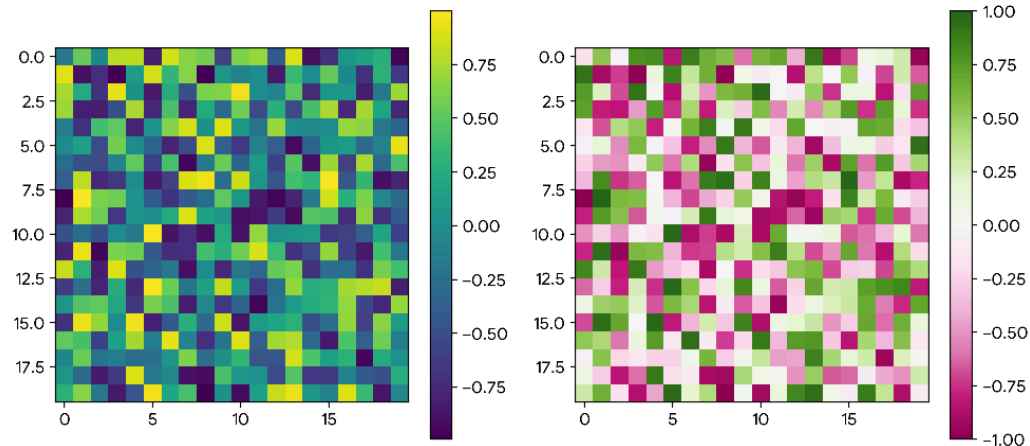
Python Code:

```
ndvi_grid = np.random.uniform(-1, 1, size=(20, 20))
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
axes[0].imshow(ndvi_grid, cmap='viridis')
axes[1].imshow(ndvi_grid, cmap='PiYG', vmin=-1, vmax=1)
plt.savefig('p7.png', dpi=200)
plt.close()
```

R Code:

```
df_sat <- expand.grid(X=1:20, Y=1:20); df_sat$ndvi <- runif(nrow(df_sat), -1, 1)
p1 <- ggplot(df_sat, aes(X, Y, fill=ndvi)) + geom_tile() + scale_fill_viridis_c()
p2 <- ggplot(df_sat, aes(X, Y, fill=ndvi)) + geom_tile() + scale_fill_gradient2(low="purple", mid="white",
high="green")
p1 + p2
```

Python Output Graph for the Scenario:



8. Scenario Title: Cryptocurrency Market Volatility

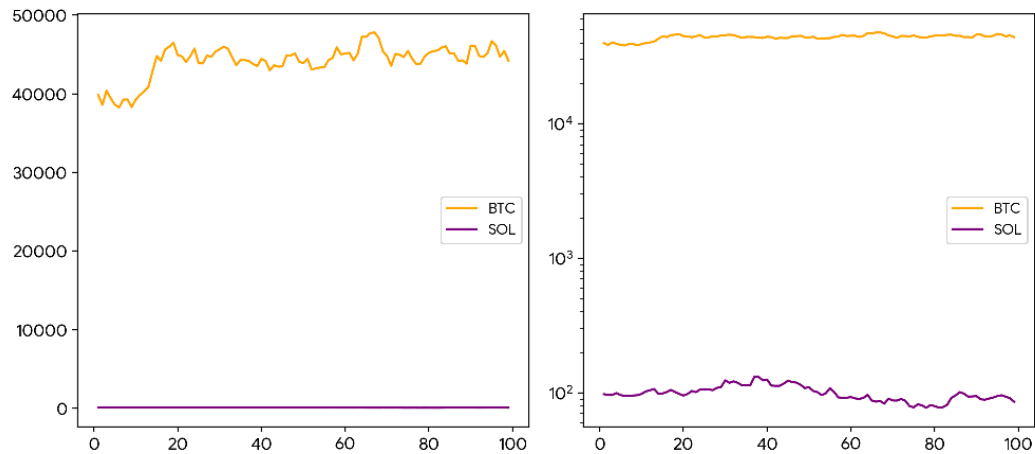
Python Code:

```
days = np.arange(1, 100)
btc = 40000 * np.exp(np.cumsum(np.random.normal(0, 0.02, 99)))
sol = 100 * np.exp(np.cumsum(np.random.normal(0, 0.05, 99)))
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
axes[0].plot(days, btc, color='orange'); axes[0].plot(days, sol, color='purple')
axes[1].plot(days, btc, color='orange'); axes[1].plot(days, sol, color='purple')
axes[1].set_yscale('log')
plt.savefig('p8.png', dpi=200)
plt.close()
```

R Code:

```
df_crypto <- data.frame(Day=1:100, BTC=cumprod(1 + rnorm(100, 0, 0.02))*40000, SOL=cumprod(1 +
rnorm(100, 0, 0.05))*100)
p1 <- ggplot(df_crypto, aes(x=Day)) + geom_line(aes(y=BTC), color="orange") + geom_line(aes(y=SOL),
color="purple")
p2 <- p1 + scale_y_log10()
p1 + p2
```

Python Output Graph for the Scenario:



9. Scenario Title: Marathon Runner Performance Analysis

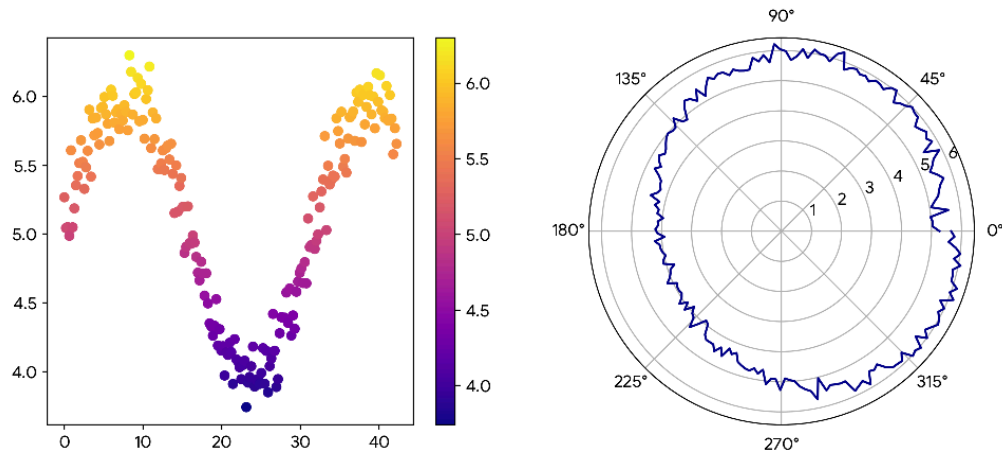
Python Code:

```
distance = np.linspace(0, 42.2, 200)
pace = 5 + np.sin(distance/5) + np.random.normal(0, 0.15, 200)
fig = plt.figure(figsize=(10, 4.5))
ax1 = fig.add_subplot(121)
ax1.scatter(distance, pace, c=pace, cmap='plasma')
ax2 = fig.add_subplot(122, projection='polar')
ax2.plot(np.linspace(0, 2*np.pi, 200), pace, color='darkblue')
plt.savefig('p9.png', dpi=200)
plt.close()
```

R Code:

```
df_runner <- data.frame(distance = seq(0, 42.2, length.out=100), pace = 5 + sin(seq(0, 42.2,
length.out=100)/5) + rnorm(100, 0, 0.1))
p1 <- ggplot(df_runner, aes(distance, pace, color=pace)) + geom_point() + scale_color_viridis_c()
p2 <- ggplot(df_runner, aes(distance*8.5, pace)) + geom_line(color="darkblue") + coord_polar()
p1 + p2
```

Python Output Graph for the Scenario:



10. Scenario Title: COVID-19 Vaccination Campaign Dashboard

Python Code:

```
districts = [f"Dist {i+1}" for i in range(5)]
vax_rates = [88, 45, 92, 61, 34]
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
axes[0].bar(districts, vax_rates, color=['green' if r >= 70 else 'red' for r in vax_rates], edgecolor='k')
axes[1].bar(districts, vax_rates, color=['#0072B2' if r >= 70 else '#E69F00' for r in vax_rates], edgecolor='k')
plt.savefig('p10.png', dpi=200)
plt.close()
```

R Code:

```
df_vax <- data.frame(District=paste("Dist", 1:5), Rate=c(85, 45, 92, 64, 25))
p1 <- ggplot(df_vax, aes(District, Rate, fill=Rate >= 70)) + geom_col() + scale_fill_manual(values=c("red",
"green"))
p2 <- ggplot(df_vax, aes(District, Rate, fill=Rate >= 70)) + geom_col() +
scale_fill_manual(values=c("#E69F00", "#0072B2"))
p1 + p2
```

Python Output Graph for the Scenario:

